Understanding a Spiking Neuron: A Tutorial on Leaky Integrate-and-Fire Dynamics and Implementation

André Furlan
Rua Cristóvão Colombo 2265, Jd Nazareth, 15054-000, São José do Rio Preto - SP, Brazil.
Instituto de Biociências, Letras e Ciências Exatas, Unesp - Univ Estadual Paulista (São Paulo State University)
São José do Rio Preto, Brazil
Email: ensismoebius@gmail.com

***Abstract*—**
The growing demand for increased processing power with deeper neural networks results in escalating energy consumption. Spiking Neural Networks (SNNs) emerge as efficient alternative to conventional neural networks, especially when implemented on neuromorphic hardware, which mimics the operating principles of biological neurons to reduce energy usage. Although frameworks like SNNTorch provide ready-to-use tools, understanding the theoretical foundations and practical implementation of SNNs remains essential, particularly for deployment in resource-constrained environments or on specialized architectures where such frameworks may not be feasible. This tutorial presents the fundamental concepts of spiking neurons and guides the reader through their implementation.

***Index Terms*—Spiking Neural Networks, SNN, LIF**

I. Introduction

Readers primarily interested in implementation details may refer to subsection II-B2.

This tutorial assumes prior familiarity with neural networks and their underlying mechanisms.

Standard neural networks (NNs), herein defined as non-spiking, require activation of all neurons during both forward and backward propagations. This means that every unit in the network must perform computations at every time step, leading to significant power consumption [1]. In an era where responsible use of energy resources is essential, a critical question arises: How can one design NN models that are energy-efficient enough to be deployed on portable or resource-constrained devices?

Spiking Neural Networks (SNNs) offer a compelling answer to this problem. As the third generation of neural networks, SNNs encode information using discrete spike events and update neuron states only when necessary. This event-driven paradigm mimics biological neuronal dynamics and allows for sparse computations, which are particularly advantageous when implemented on neuromorphic hardware. Such systems can achieve high energy efficiency by leveraging the inherent sparsity of spike-based communication.

Despite the availability of tools such as SNNTorch for SNN simulation and training, a comprehensive understanding of their theoretical foundations and low-level implementation remains essential — especially in scenarios where existing frameworks are incompatible with specialized hardware or minimal runtime environments.

This paper aims to provide a detailed exposition of the theoretical principles of spiking neurons, culminating in a practical implementation guide

II. Biological and computational background

Biological sensory systems convert external stimuli — such as light, odors, touch, and taste — into discrete electrical events known as **spikes**. A spike is a voltage pulse that conveys information through the nervous system [5]. These signals are propagated along neural pathways to be processed, ultimately generating behavioral or physiological responses to the environment.

A biological neuron emits a spike only when the accumulated excitatory input at the soma exceeds a certain threshold. In the absence of such stimulation, the neuron remains inactive. This event-driven behavior makes biological neurons highly efficient in terms of energy consumption and computational resources.

Spiking Neural Networks (SNNs) aim to emulate some of these advantages by processing **spikes** at the input, hidden, and output layers. SNNs can also accept continuous-valued inputs retaining their event-driven properties and energy efficiency due to sparse activation patterns.

SNNs are **not** detailed simulations of biological neurons. Instead, they approximate specific computational characteristics inspired by neurobiology. More biologically faithful models - such as the **Hodgkin–Huxley neuron** [2] and those explored in [4], which incorporate dendritic nonlinearities and other advanced neuronal features - have shown remarkable results in classification tasks, but at the cost of greater computational complexity.

As illustrated in Figure 1, SNN neurons exhibit strong noise tolerance when appropriately parameterized, functioning as **low-pass filters**. They can still generate reliable spikes under considerable interference. Moreover, SNN neurons are inherently time-sensitive, making them especially effective for processing temporal or sequential data streams [1].

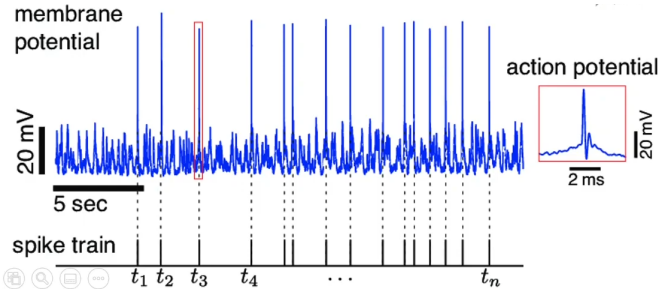

Fig. 1: Spikes from a noisy signal: The noisy signal in blue is filtered generating sparse spikes. Source [3]

A. *Leaky Integrate and Fire Neuron*

The focus of this work is the *Leaky Integrate-and-Fire* (LIF) neuron due to its simplicity, computational efficiency, and its strong generalization performance across many tasks [3].

Unlike a standard artificial neuron, LIF integrates incoming weighted inputs over time with a *leakage* mechanism that continuously reduces the membrane potential, simulating the passive decay observed in biological membranes. Only when the membrane potential crosses a threshold, does the neuron emit a spike, after which the potential resets or decreases.

LIF behaves much like Resistor-Capacitor circuit like as illustrated in Figure 2.

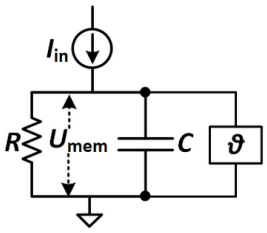

Fig. 2: RC model with switch: R represents the leakage resistance, I_{in} the input current, C the membrane capacitance, U_{mem} the membrane potential (i.e., the accumulated charge) and ϑ a switch that lets the capacitor discharge (i.e. emit a spike) when a voltage threshold is reached. Source: [1]

Spikes are represented as **sparsely** distributed **ones** in a train of **zeros**, as illustrated in Figure 3 and Figure 4. This approach simplify the models and reduces the computational power and needed storage to run a SNN.

As a result of the aforementioned characteristics, information in SNNs is encoded in terms of the *timing* and/or *rate* of spikes. This enables effective processing of temporal data streams but imposes limitations when dealing with static data like images which need some preprocessing in order to be better interpreted by a spiking neural network.

These preprocessing steps may consist of repeated copies of the input data for temporal reinforcement, or of a progressive decomposition of the input information, enabling its presentation in temporally distributed segments.

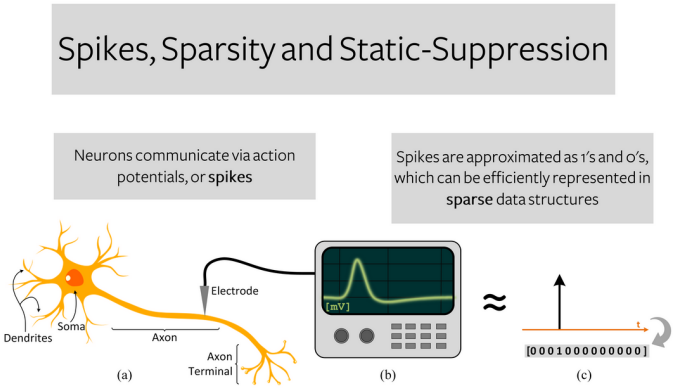

Fig. 3: Sparsity on Spike Neuron Networks. Source: [1]

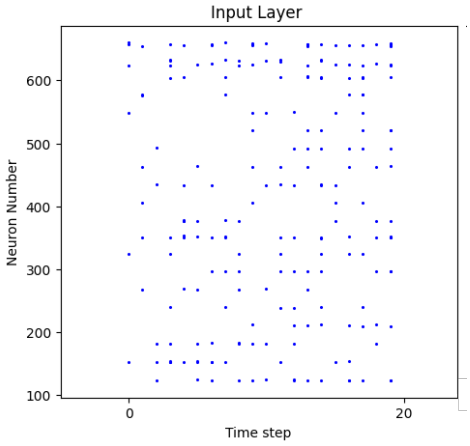

Fig. 4: Sparse activity of a SNN: The horizontal axis represents the time step of some data being processed the vertical are the SNN neuron number. Note that, most of the time, very few neurons gets activated. Source: The author.

B. *Mathematical Model of the LIF Neuron inner workings*

The first subsection presents an analytical study of the *Resistor-Capacitor (RC) circuit model*, which governs the membrane potential dynamics of the LIF neuron, including both charging and discharging phases. Corresponding implementations and plots are provided. Building on this foundation, the second subsection focuses on implementing the complete LIF model, incorporating threshold mechanisms and potential resets.

*1) **Membrane Charging and Discharging**:* As shown in Figure 2, a LIF neuron can be modeled as an RC circuit with a switch. By isolating the RC circuit, as illustrated in Figure 5, one can analyze the dynamics of membrane currents and potential governed by the equations described in [1] and shown bellow:

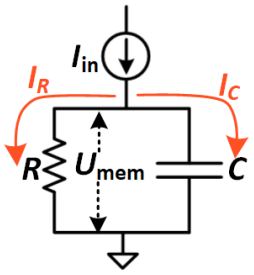

Fig. 5: RC model for currents: $I_{in}(t) = I_R(t) + I_C(t)$

Let's consider Q as a measurement of electrical charge and $V_{mem}(t)$ is the membrane potential in a certain time t than the neuron capacitance C is given by Equation 1.

$$C = \frac{Q}{V_{mem}(t)} \tag{1}$$

Therefore the neuron charge can be expressed as Equation 2.

$$Q = C.V_{mem}(t) \tag{2}$$

To know how the charge changes according to the time (a.k.a. current) we can differentiate Q as in Equation 3. This is the expression of the electric current in the capacitive part of the neuron I_C.

$$I_C(t) = \frac{dQ}{dt} \implies \boxed{I_C(t) = C.\frac{dV_{mem}(t)}{dt}} \tag{3}$$

Then to calculate the total current passing by the resistive part R of the circuit according to Ohm's law:

$$V_{mem}(t) = R.I_R(t) \implies \boxed{I_R(t) = \frac{V_{mem}(t)}{R}} \tag{4}$$

Than considering that the total current do not change, as seen in Figure 5, then the total input current I_{in} of the neuron can be expressed as in Equation 5.

$$I_{in}(t) = I_R(t) + I_C(t) \implies \boxed{I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt}} \tag{5}$$

Therefore, to describe the passive membrane we got a linear Equation 6.

$$\begin{gathered} I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt} \implies \\ I_{in}(t) - \frac{V_{mem}(t)}{R} = C.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{R.I_{in}(t) - V_{mem}(t) = R.C.\frac{dV_{mem}(t)}{dt}} \end{gathered} \tag{6}$$

Defining $\tau = RC$ as the **membrane time constant**, we obtain Equation 7, which describes the RC circuit dynamics. This terminology arises from dimensional analysis: the right-hand side of the equation has units of *voltage*, whereas the left-hand side contains $\frac{dV_{mem}(t)}{dt}$, which has units of *voltage per time*. To maintain dimensional consistency, RC must therefore be a unit of time.

$$\begin{gathered} R.I_{in}(t) - V_{mem}(t) = R.C.\frac{dV_{mem}(t)}{dt} \implies \\ R.I_{in}(t) - V_{mem}(t) = \tau.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t)} \end{gathered} \tag{7}$$

To model the **potential decay**, we set $I_{in}(t) = 0$ (i.e., no input) and assume $\tau = RC$ as constant with an initial membrane potential $V_{mem}(0) > 0$. Under these conditions, the membrane voltage follows an exponential decay, as shown in Equation 8.

$$\begin{gathered} \tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) \implies \\ \tau.\frac{dV_{mem}(t)}{dt} = R.0 - V_{mem}(t) \implies \\ \tau.\frac{dV_{mem}(t)}{dt} = -V_{mem}(t) = \\ e^{\ln(V_{mem}(t))} = e^{-\frac{t}{\tau}} = \\ \boxed{V_{mem}(t) = V_{mem}(0).e^{-\frac{t}{\tau}}} \end{gathered} \tag{8}$$

In summary: In the absence of an input I_{in}, the membrane potential decays exponentially as illustrated in Figure 6 and implemented in Listing 1.

```python
def lifDecay(V_mem, t=1, R=5, C=1):
  tau = R*C

  # e^(-dt/tau)
  decay_rate = np.exp(-t/tau)

  # V_mem(t) = V_mem(0) * e^(-t/tau)
  V_mem = V_mem * decay_rate

  return V_mem
```

Listing 1: Python implementation of the deacreasing action potential behavior of a LIF: **V_mem**: Membrane potential at the current time step; **t=1**: simulation time step; **R=5**: membrane resistance; **C=1**: membrane capacitance; **tau = R * C**: membrane time constant (i.e., the characteristic decay time of the membrane potential); **decay_rate=-t/tau**: The rate by which the membrane voltage decays for each time step.

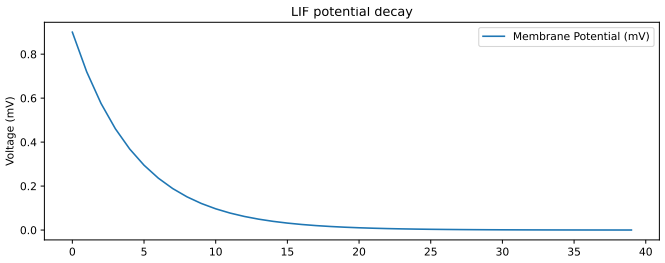

Fig. 6: Membrane potential decaying when $I_{in} = 0$: Time is represented by the horizontal axis. Neuron charge in Volts is the vertical axis. Source: The author

With the results from Equation 7 it is possible to calculate the action potential increasing as seen in Equation 9.

$$\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) =$$
$$\frac{dV_{mem}(t)}{dt} + \frac{V_{mem}(t)}{\tau} = \frac{R.I_{in}(t)}{\tau} \implies$$
$$\text{Integrating factor: } e^{\int \frac{1}{\tau} dt} = e^{\frac{t}{\tau}} \implies \quad (9)$$
$$V_{mem}(t) = \frac{1}{e^{\frac{t}{\tau}}}.\left(\int \frac{R.I_{in}(t)}{\tau}.e^{\frac{t}{\tau}} dt\right) \therefore$$
$$\boxed{V_{mem}(t) = I_{in}(t).R(1 - e^{\frac{-t}{\tau}})}$$

Note that, as seen in Figure 7 and implemented in Listing 2, action potentials also increases exponentially.

```python
def lifIncrease(V_mem, t=1, I_in=1, R=5, C=1):
  tau = R * C

  # e^(-dt/tau)
  decay_rate = np.exp(-t/tau)

  # V_mem(t)=I_in(t).R.(1 - e^(-t/tau))
  V_mem = V_mem + I_in * R * (1 - decay_rate)

  return V_mem
```

Listing 2: Python implementation of the increasing action potential behavior of a LIF: **V_mem**: Membrane potential at the current time step; **t=1**: simulation time step; **R=5**: membrane resistance; **C=1**: membrane capacitance; **tau = R * C**: membrane time constant (i.e., the characteristic decay time of the membrane potential); **decay_rate=-t/tau**: The rate by which the membrane voltage decays for each time step.

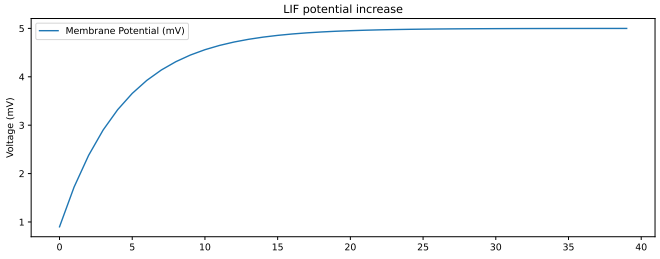

Fig. 7: Membrane potential increasing when setting I_in = 1 in Listing 2: Time is represented by the horizontal axis. Neuron charge in Volts is the vertical axis. Source: The author

*2) **Implementation with Resets and Thresholds**:* Starting from Equation 7, we apply the forward Euler method to numerically solve for successive values of V_{mem}. This enables the construction of a fully functional LIF model.

$$\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) \implies$$
$$\tau.\frac{V_{mem}(t+\Delta t) - V_{mem}(t)}{\Delta t} = R.I_{in}(t) - V_{mem}(t) \implies$$
$$V_{mem}(t+\Delta t) - V_{mem}(t) = \frac{\Delta t}{\tau}(R.I_{in}(t) - V_{mem}(t)) \implies$$
$$\boxed{V_{mem}(t+\Delta t) = V_{mem}(t) + \frac{\Delta t}{\tau}(R.I_{in}(t) - V_{mem}(t))} \quad (10)$$

Now, introducing a **threshold** that triggers a reset of the membrane potential—either by resetting to zero or by subtracting the threshold value—allows us to model the complete charge and discharge behavior of the LIF neuron. This behavior is illustrated in Figure 8 and implemented in Listing 3.

```python
def lif(V_mem, dt=1, I_in=0.5, R=5, C=1, V_thresh =
    2, reset_zero = True):
  tau = R*C

  # Default: No spike
  spike = 0

  # V_mem(t+dt)=V_mem(t)+(dt/tau)*(R*I_in-V_mem(t))
  V_mem = V_mem + (dt / tau) * (I * R - V_mem)

  # Threshold
  if V_mem > V_thresh:

    # Emit a spike
    spike = 1

    if reset_zero:
      V_mem = 0
    else:
      V_mem = V_mem - V_thresh

  return V_mem, spike
```

Listing 3: Python implementation of the full action potential simulation of a LIF: **V_mem**: current membrane potential; **dt=1**: simulation time step; **I_in=0.5**: input current (assumed constant during dt); **R=5**: membrane resistance; **C=1**: membrane capacitance; **V_thresh=2**: firing threshold, when V_mem > V_thresh, the neuron "spikes"; **reset_zero=True**: determines if the neuron resets to zero after spiking; **tau = R * C**: membrane time constant (i.e., the characteristic decay time of the membrane potential);

This implementation already fulfills the functional requirements of a LIF neuron. However, it still involves several hyperparameters, many of which are unnecessary. From this point onward, we shift the focus from biological accuracy to functional simplicity, replacing or eliminating these hyperparameters as needed.

The first hyperparameter that can be replaced is Δt. In our context—where time is discretized and biological accuracy is not required—it is reasonable to assume a minimum time step of 1 (Equation 11) for the neuron to decide whether or not to emit a spike.

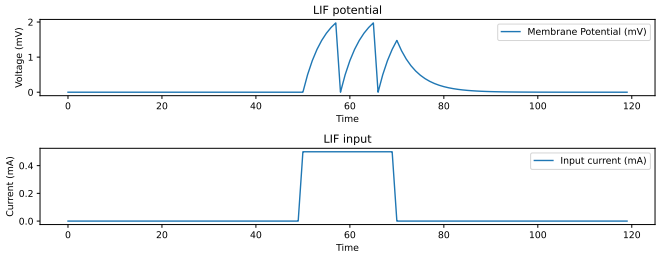

Fig. 8: Plot of the full simulated LIF: a 0.5 mA input current was applied from time step 51 to 70. Note that the neuron spikes twice, but it does not accumulate enough charge to generate a third spike. Afterward, the membrane potential decays back to zero.

$$V_{mem}(t+\Delta t) = V_{mem}(t) + \frac{\Delta t}{\tau}(R.I_{in}(t) - V_{mem}(t)) \implies$$
$$\boxed{V_{mem}(t+1) = V_{mem}(t) + \frac{1}{\tau}(R.I_{in}(t) - V_{mem}(t))} \tag{11}$$

Another simplification involves the membrane time constant: since $\tau = R.C$, we can arbitrarily set either R or C to 1, making the computation of τ redundant. The resulting value can still be referred to as τ for consistency as seen in Equation 12.

$$\begin{aligned} &\tau = R.C \implies \\ &\tau = 1.C \implies \\ &\boxed{\tau = C} \end{aligned} \tag{12}$$

After some refactoring and by setting $I_{in}(t) = 0$ (i.e., no input) in Equation 11, we can analyze how the membrane potential decays — more specifically, its decay rate β, as expressed in Equation 13.

$$\begin{aligned} &V_{mem}(t+1) = V_{mem}(t) + \frac{1}{\tau}(\cancel{R}^{1}.I_{in}(t) - V_{mem}(t)) \implies \\ &V_{mem}(t+1) = V_{mem}(t) + \frac{1}{\tau}.I_{in}(t) - \frac{1}{\tau}.V_{mem}(t) \implies \\ &V_{mem}(t+1) = \frac{1}{\tau}.I_{in}(t) - \frac{1}{\tau}.V_{mem}(t) + V_{mem}(t) \implies \\ &V_{mem}(t+1) = \frac{1}{\tau}.I_{in}(t) + V_{mem}(t)\left(1 - \frac{1}{\tau}\right) \implies \\ &V_{mem}(t+1) = \frac{1}{\tau}.\cancel{I_{in}(t)}^{0} + V_{mem}(t)\left(1 - \frac{1}{\tau}\right) \implies \\ &V_{mem}(t+1) = V_{mem}(t)\left(1 - \frac{1}{\tau}\right) \implies \\ &\frac{V_{mem}(t+1)}{V_{mem}(t)} = 1 - \frac{1}{\tau} = \beta \implies \qquad \boxed{\beta = 1 - \frac{1}{\tau}} \end{aligned} \tag{13}$$

Then, by substituting β into Equation 13, we obtain a new LIF equation that no longer depends explicitly on the parameters τ, R, or C. These parameters can, be replaced by the single parameter β.

$$\begin{aligned} &V_{mem}(t+1) = \frac{\overset{1-\beta}{\cancel{1}}}{\tau}.I_{in}(t) + V_{mem}(t)\left(1 - \frac{\overset{\beta}{\cancel{1}}}{\tau}\right) \implies \\ &V_{mem}(t+1) = (1-\beta).I_{in}(t) + V_{mem}(t)\beta \implies \\ &V_{mem}(t+1) = \beta V_{mem}(t) + (1-\beta).I_{in}(t) \end{aligned} \tag{14}$$

Going further, in Equation 14, as we know, $I_{in}(t)$ is the input to the neuron, and $1-\beta$ serves as the **weighting factor**, commonly denoted by X and W, respectively. Which finally results in the simplified form of LIF shown in Equation 15 and coded in Listing 4.

$$\boxed{V_{mem}(t+1) = \beta V_{mem}(t) + W.X(t)} \tag{15}$$

But wait: Why doesn't $\beta V_{mem}(t)$ change? This term remains because it encapsulates the neuron's **memory component**, as shown in Equation 16 and detailed in Table I [2]. It governs how much of the past membrane potential is preserved at each time step, directly modulated by the decay factor β which is now independent of W.

$$\begin{aligned} \lim_{\beta \to 0} &\implies \text{short memory neuron, high adaptability} \\ \lim_{\beta \to 1} &\implies \text{long memory neuron, low adaptability} \\ \beta = 1 &\implies \text{perfect integrator, no decay} \\ \beta = 0 &\implies \text{just reacts to current input, no memory} \end{aligned} \tag{16}$$

β	Memory time	Noise response	Ideal input type
Low (< 0.3)	Short	High	Strong and sudden
Medium (0.5–0.9)	Moderate	Balanced	Patterns with some persistence
High (> 0.95)	Long	Low	Weak and continuous stimuli

TABLE I: Practical consequences of varying β in the LIF model

```python
def lif(V_mem, X, W, beta, V_thresh=2):

  # if V_mem > V_thresh, spk=1, else, 0
  spk = (V_mem > V_thresh)

  # V_mem(t + 1) = beta * V_mem(t) + W * X(t)
  # - threshold (if spiked)
  V_mem = beta*V_mem + W*X - spk*V_thresh

  return spk, V_mem
```

Listing 4: Python implementation of the simplified full action potential simulation of a LIF: **V_mem**: current membrane potential; **X**: input current (assumed constant during dt); **V_thresh=2**: firing threshold, when V_mem > V_thresh, the neuron "spikes";

Note that Listing 4 does not implement a reset-to-zero mechanism when the neuron spikes. This omission is intentional to avoid branching in the CPU or GPU execution pipeline. If a reset is required, it can be easily added without altering the core dynamics as seen in the Listing 3.

C. *Training*

Until now just the *forward* pass has been shown, it's time to describe the backwards.

As observed, the behavior of an SNN neuron closely resembles a **step function**—it either fires or not. Consequently, gradient descent-based methods are, in principle, inapplicable, since such functions are **not** differentiable [5]. While this is not necessarily a problem—for example, ReLU is also non-differentiable at zero, yet still usable due to its piecewise linearity—the situation with spiking neurons is more problematic: the neuron remains inactive for most of the time (a.k.a. dead neuron) and then emits a spike, producing an infinite gradient. This is uninformative, as it does not yield a usable gradient curve to minimize.

But there are some insights out there that may shed some light on this subject: While some *in vivo/in vitro* observations show that brains, in general, learn by strengthening/weakening and adding/removing synapses or even by creating new neurons or other cumbersome methods like RNA packets, there are some more straightforward ways like the ones listed bellow [5]:

- **surrogate gradient descent(chosen)**: Approximates the step function by using another mathematical function, which can be minimized, in order to train the network. These approximations are used only in the backward pass, while keeping the step function behavior in the forward pass [5].
- **Spike Timing-Dependent Plasticity (STDP)**: If a pre-synaptic neuron fires **before** the post-synaptic one, there is a strengthening in connection, but if the post-synaptic neuron fires before, then there is a weakening.
- **evolving algorithms**: Use the selection of the fittest throughout many generations of networks.
- **reservoir/dynamic Computing**: **Echo state networks** or **Liquid state machines** respectively.

We adopt the method of **surrogate gradient descent**, using the *HardTanh* function as the chosen approximation for the activation. Its definition is given in Equation 17, and its derivative in Equation 18.

$$hardtanh(x) = \begin{cases} maxVal, & x > maxVal \\ minVal, & x < minVal \\ x, & otherwise \end{cases} \tag{17}$$

$$\frac{d}{dx} hardtanh(x) = \begin{cases} 0, & x > maxVal \\ 0, & x < minVal \\ 1, & otherwise \end{cases} \tag{18}$$

Given the nature of the SNN outputs (*spiking*: output = 1, *no spiking*: output = 0) the *maxVal* is set 1 and *minVal* is set to 0 resulting in Listing 5, with the corresponding plot shown in Figure 9.

```python
# Define the HardTanh function
def hardtanh(x):
  return ((x > 0) * x) * (x < 1) + (x >= 1)

# Define the derivative of the HardTanh function
def d_hardtanh(x):
  return (x > 0) * (x < 1)
```

Listing 5: Python implementation of hardTanh and the derivative of hardTanh

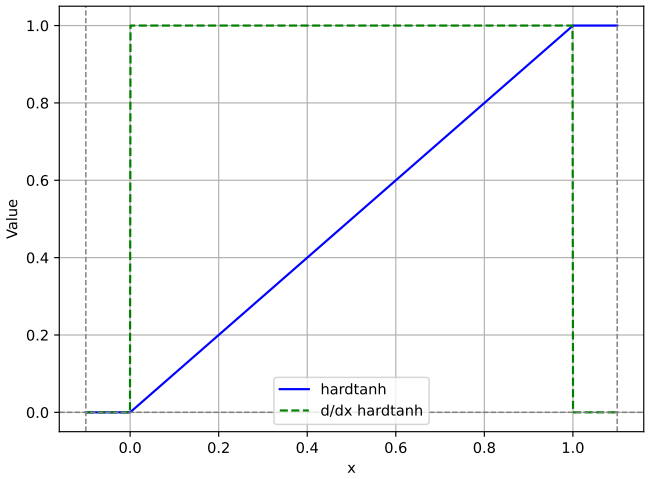

Fig. 9: The HardTanh function is shown in blue, and its derivative is represented by the green dashed line.

Now we have defined all needed concepts needed for a full LIF neuron it is possible to implement the *forward* and *backward* parts needed for the assembly of a full spiking neural network as shown in Listing 6.

```python
import numpy as np

class SpikeNeuron:
  def __init__(self, beta, V_thresh=1.0):

    # beta: decay factor for membrane potential
    self.beta = beta
    # V_thresh: threshold for spike generation
    self.V_thresh = V_thresh

    # Membrane potential, initialized to None
    # It will be initialized to zeros in the first forward pass
    # This allows the neuron to adapt to the input shape dynamically
    # and avoids the need for a fixed input shape at initialization.
    self.V_mem = None

    # Store the last membrane potential before spike generation
    # This is used for surrogate gradient calculation in the backward pass
    self.last_Vmem_before_spike = None

  def d_hardtanh(x):
    """
    Surrogate gradient for hardtanh activation function.
    This is used for backpropagation through the spike neuron.
    """
    return ((x > 0.0) & (x < 1.0)).astype(float)

  def forward(self, I_t):
    """
    I_t: input current
    This method computes the spike output based on the input current and the membrane potential.
    It updates the membrane potential and generates spikes based on the threshold.
    """

    # Initialize membrane potential if it is None
    if self.V_mem is None:
    self.V_mem = np.zeros_like(I_t)

    # Update the membrane potential based on the input current and previous state
    spk = (self.V_mem > self.V_thresh).astype(float)

    # Store the last membrane potential before spike generation
    # This is used for surrogate gradient calculation in the backward pass
    self.last_Vmem_before_spike = self.V_mem.copy()

    # Update the membrane potential using the decay factor and input current
    self.V_mem = self.beta * self.V_mem + I_t - spk * self.V_thresh

    # Return the spike output
    return spk

  def backward(self, grad_spike_out):
    """
    grad_spike_out: gradient from the next layer (shape same as forward output)
    Returns: gradient w.r.t. input (to propagate to previous layers)
    """

    # Surrogate gradient for the hardtanh activation function
    surrogate_grad = self.d_hardtanh(self.last_Vmem_before_spike - self.V_thresh)

    # Compute the gradient w.r.t. input current
    grad_input = grad_spike_out * surrogate_grad

    return grad_input
```

Listing 6: LIF class with forward and backward methods

References

[1] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.

[2] W. Gerstner, W.M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.

[3] Dan Goodman, Tomas Fiers, Richard Gao, Marcus Ghosh, and Nicolas Perez. Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022, September 2022.

[4] Ilenna Simone Jones and Konrad Paul Kording. Can single neurons solve mnist? the computational power of biological dendritic trees, 2020.

[5] Nikola K Kasabov. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. Springer, 2019.